

Ej. 1	Ej. 2	Ej. 3	Nota
B	B	B	16

Matic

Aclaraciones

- Anote apellido, nombre, LU y numere *todas* las hojas entregadas.
- Cada ejercicio será calificado con una de las siguientes tres notas: Bien, Regular o Mal. La división de los ejercicios en incisos es meramente orientativa. Los ejercicios se calificarán globalmente.
- El parcial **no es a libro abierto**, sólo se podrán utilizar las hojas que son entregadas como enunciado.
- Un resultado sin suficiente justificación equivale a un ejercicio no resuelto.
- El parcial se aprueba con al menos dos ejercicios Bien y uno Regular. Para obtener un Regular es necesario demostrar conocimientos sobre el tema del ejercicio.

Ejercicio 1 Implemente la función `rec` en el lenguaje ensamblador RISC V de forma recursiva, respete la convención de llamada presentada en la materia, explique el uso que le dará a cada registro y cómo se asegura que sus valores se preservan antes y después de cada llamada a función. No deben implementar la versión iterativa, debe ser recursiva, en caso contrario se considera desaprobado el ejercicio. Pueden usar la instrucción `mul`.

$$rec(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ 2 * rec(n - 1) + 3 * rec(n - 2) & \text{si } n > 1 \end{cases}$$

Guía de resolución (opcional):

- Escriba una versión de pseudocódigo.
- Transforme cada caso a su equivalente de operaciones atómicas (descomponga las operaciones lógicas, aritméticas y llamadas a función).
- Identifique los registros a emplear para cada dato.
- Si debe preservar algún registro para respetar la convención, indique qué mecanismo utilizará.
- Defina un flujo de ejecución tentativo.

Importante: Justifique sus respuestas.

Ejercicio 2 Para entender el balance del sistema previsional contamos con un arreglo que contiene una estructura de aportantes con su edad, cantidad de años de aportes y pesos que aportaron al sistema.

Se cuenta con un arreglo **aportantes** que contiene una cantidad fija cuyo tamaño está definido por la constante `largo`. Cada aportante se define como una estructura de cuatro bytes, donde el primer byte determina la **edad** como entero sin signo, el segundo byte determina la cantidad de años de aportes como un entero sin signo y los siguientes dos bytes (half word) determinan la cantidad de pesos que aportaron al sistema.

Escriba un programa que calcule el aporte promedio para las personas cuya edad sea mayor o igual a 65 años.

Ejemplo:

Dirección
aportantes

0x0000	0x0x0001	0x0002	0x0004	0x0005	0x0006
66	43	5000	32	4	-500
0x0008	0x0x0009	0x000A	0x000C	0x000D	0x000E
70	40	-1000	25	5	200

Noten que las direcciones están en hexadecimal (por eso el prefijo 0x) y los valores en decimal. En este caso tendríamos que sumar los aportes de del primer y tercer elemento por tener edades mayores a 65 y calcular el promedio que sería de 2000 pesos.

Esqueleto de programa:

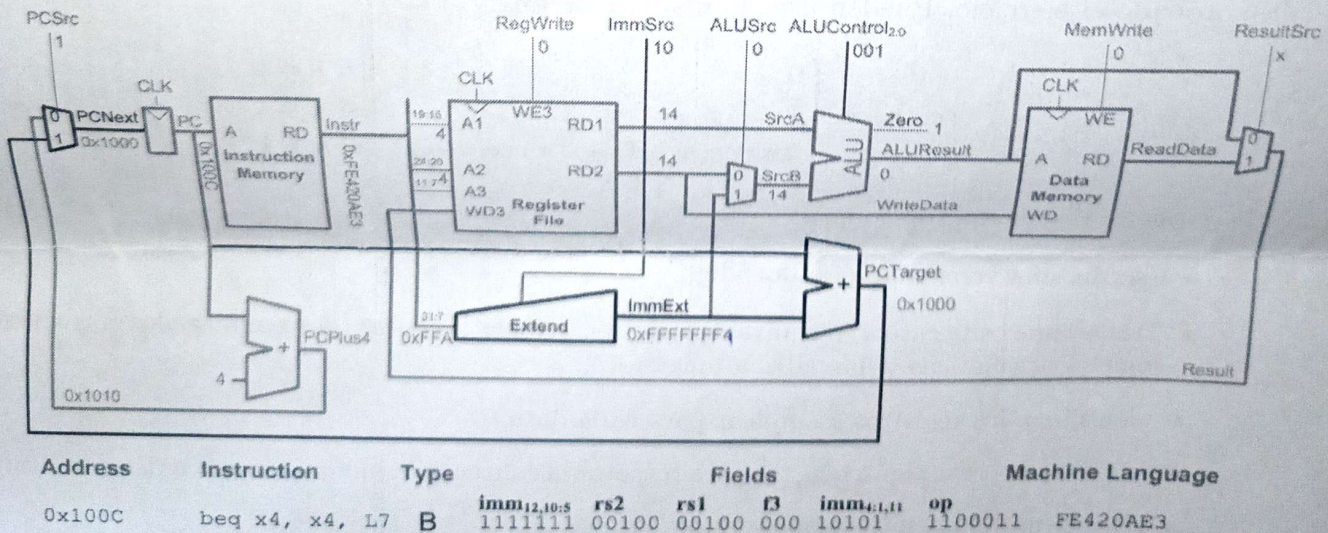
```

1 .data:
2 aportantes:
3     .byte 66, 43
4     .half 5000
5     .byte 32, 4
6     .half -500
7     .byte 70, 40
8     .half -1000
9     .byte 25, 5
10    .half 200
11    largo: .byte 4
12
13 .text:
14 # Escribir el programa aca.

```

Importante: Justifique sus respuestas.

Ejercicio 3 Dado el siguiente datapath de la instrucción beq que se detalla, explique cómo se calcula la salida del extensor de signo para este caso. Justifique cual sería la dirección de la próxima instrucción a ejecutar. ¿Qué función cumplen las señales PCSrc, ImmSrc, ALUSrc, ALUControl_{2:0}? ¿Qué componente define los valores de estas señales? **Importante:** Justifique sus respuestas.



ej 1) rec: #a0 = n nuevo stack
pointer.

```

addi sp, sp, -16
sw ra, 0(sp) # Tomo espacio en el stack y
sw s0, 4(sp) # guardo en memoria los valores
sw s1, 8(sp) # que debo preservar (ya que los
               # reescribo en rec).

# li a1, 1
ble a0, a1, fin # si a0 ≤ a1 salta, si n ≤ 1 devuelvo n.
mv s0, a0 # s0 = n : lo guardo para mantenerlo luego
addi a0, a0, -1 # a0 = n - 1 en s0 de rec
jal rec # a0 = rec(n-1)
slli s1, a0, 1 # s1 = 2 * a0 : lo guardo para mantenerlo
addi a0, s0, -2 # a0 = n - 2 en s1 luego de rec
jal rec # a0 = rec(n-2)
mul s1, s1, s1
mul a0, a0, s1
add a0, s1, a0 # a1 = 3
               # a0 = 3 * rec(n-2)
               # a0 = 2 * rec(n-1) + 3 * rec(n-2)

Fin:
lw s1, 8(sp)
lw s0, 4(sp)
lw ra, 0(sp)
addi sp, sp, 16
ret

```

Recupero valores que debo preservar y restauro stack pointer.

Como lo expliqué en los comentarios, uso registros que se deben preservar en la función, pero como los guardo en memoria al principio y al final los recupero, preservó los valores que tenían antes de entrar a la función (los registros son ra, s0, s1). El a0 por convención tiene el valor de retorno de la función y el que respeta esto en la función; además uso a0 y a1 dentro de la función, pero estos no los guardo porque son volátiles. Como dije antes, el resultado estará en a0 al terminar la función como dicta la convención los valores que están en registros volátiles y quiero mantener, los guardo en registros preservables para asegurarme que la función llamada no los va a modificar (por convención).

Ej 2) Asumo que tengo aportantes y largo como constantes en .data:

Aporte prom:

```
addi sp, sp, -16 # Guardo ra en stack
sw ra, 0(sp)      (por las dudas) → No es necesario
```

```
li to, 0 # to = cant personas > 65 la edad
li t1, 0 # t1 = suma aportes
li t2, 65 # t2 = 65
```

```
la a0, aportantes # Cargo puntero al arreglo en a0
la a1, largo # Cargo dir del valor largo
lbu a1, 0(a1) # Recupero valor de largo de memoria (lo tomé como bu
```

```
while: # Si largo a1 == 0 termina
begz a1, finWhile
lbu a2, 0(a0) # porque en el ejemplo lo definen como byte y las longitudes siempre son positivas). en a1
```

```
# a2 = edad
blt a2, t2, noSuma* # Si edad < 65 no cuento nada
addi to, to, 1 # cant ++
addi a0, a0, 2 # nuevo puntero a pesos (2 bytes)
lh a2, 0(a0) # a2 = pesos
add t1, t1, a2 # agrego pesos a aportes totales.
```

```
addi a1, a1, -1 # largo --
addi a0, a0, 2 # nuevo puntero al siguiente
j while # aportante
```

← finWhile:

```
div a0, t1, to # a0 = suma aportes con mas de 65 años / cant personas
```

```
lw ra, 0(sp) # Recupero ra y valor del sp.
addi sp, sp, 16 # a0 = aporte promedio de personas con edad ≥ 65
ret
```

Como no uso ningún ~~valor~~ registro preservable, no hace falta que los guarde en memoria (solo guardo ra y recupero ra).

Uso registros volátiles a0-a2 y to-t2 (los elegí casi de forma arbitraria, usando los "a" para cargar info de .data). Respeto la convención del valor de retorno ya que lo guardo en a0.

* noSuma:

```
addi a1, a1, -1 # largo --
addi a0, a0, 4 # Paso al siguiente aportante. Antes a0 estaba en el 1er byte (edad)
j while # y aportante tiene 4 bytes de tamaño
```

Debería ir acá el noSuma*, antes del finWhile y después del j while.

Ej 3). El extensor de signo en este caso está con la señal ImmSrc correspondiente a instrucciones de saltos, es decir, que tiene inmediato de 13 bits con el bit 0 sin representar (ya que siempre es 0 para los saltos porque el PC tiene direcciones pares (las instrucciones son palabras de 4 bytes)). Este inmediato se extiende primero agregando el bit 0 con el valor 0 y extendiendo el signo a 32 bits. Al agregar este último 0, ~~el byte~~ los 4 bytes menos significativos pasan de ser $1010 = A$ a ser $0100 = 4$ y por eso el resultado de extender $0xFFA$ es $0xFFFFF4$ (los F vienen de la extensión del signo negativo).

- Como bcg salta a L7 si $X_4 = X_4$, y estoy comparando el mismo registro, esta condición se cumple y el PCSrc ~~se~~ envía la señal 1 al multiplexor de PC Next, ~~de~~ restando PC Next en la dirección de la etiqueta L7 (se calcula sumando el inmediato que extendimos antes (offset) más el valor anterior del PC ~~(PC Target)~~, este nuevo valor es PC Target). Este nuevo valor de PC Next determina la dirección de la próxima instrucción a ejecutar (PC Target).

- Las señales PCSrc, ImmSrc anteriormente dije que valores tenían para esta instrucción. En general, PCSrc indica si el nuevo valor del ~~inmediato~~ PC se va a obtener de sumarle 4 al PC (señal 0) o de sumarle un inmediato (offset) arbitrario (señal 1). ImmSrc indica cómo se va a extender el inmediato y ~~cuántos~~ cuántos bits tiene el inmediato (depende del tipo de operación).

ALUSrc indica si el segundo operando del ALU viene de los registros o de un inmediato (señal 0 para registros y 1 para inmediato); en este caso la ALU realiza una operación con registros (señal 0): X_4 y X_4 .

ALUcontrol indica a la ALU qué operación debe hacer. En este caso sería una resta para hacer $X_4 - X_4$ y poder ver si se enciende el flag zero (si son iguales).

- Todas estas señales son señales de control (PCSrc, ImmSrc, ALUSrc, ALUcontrol), cuyos valores son definidos por la unidad de control.

La unidad de control es la encargada de decodificar la instrucción y a partir de eso definir los valores de las señales de control. En este caso además, ~~es~~ es la encargada de revisar si el flag zero está en uno y relear (con PCSrc) si se hace el salto.